

① Adapter Pattern :- It is used to convert one interface to another. It wraps an incompatible class and makes it usable via a desired interface.

When to use it :- When integrating
third party or legacy code with
a new System.

eg :- Old Payment System in a
New Interface

```
interface PaymentProcessor
```

```
{    void pay();
```

```
}
```

```
class LegacyBank
```

```
{    void makeTransfer()
```

```
{    S.O.P ("Bank Transfer Done"),
```

```
}
```

```
}
```

class BankAdapter implements
PaymentProcessor

```
{  
    private LegacyBank bank;  
  
    BankAdapter (LegacyBank bank)  
    {  
        this.bank = bank;  
    }  
  
    public void pay ()  
    {  
        bank.makeTransfer();  
    }  
}
```

// Usage

```
PaymentProcessor adapter = new  
BankAdapter(  
new LegacyBank());  
adapter.pay();
```

// Output : Bank Transfer
Done

o Helps with : interface compatibility between
incompatible systems.

Class LegacyBank

```
void makeTransfer  
{  
    S.O.P("Bank  
Transfer done");  
}
```

Interface
Pay()

class Bank Adapter

L.B (bank())

pay()

```
{  
    bank.makeTransfer();  
}
```

```
a =  
new BankAdapter  
(new L.B)  
a.pay();
```

① Bridge Pattern :- It is used to separate abstraction from implementations. It decouples abstraction from implementation so that they can evolve independently.

② When to use :- When you want to vary not only behaviour but also implementations (eg:- different platforms for parking displays).

eg:- Parking Displays on different
Screens

```
interface Display
```

```
{ void show (String message),  
}
```


class LEDisplay implements Display

{
 p.v show (String message)

{
 s.o.p (" L E D " + message);
}

}

class LCDisplay implements
Display

{
 p.v show (String message)

{
 s.o.p (" L C D " + message);
}

// abstract class ParkingSystem

{ Display display;

← ParkingSystem (Display display)

{ this.display = display;

}

// abstract void notifyUser(),

}

```
class ExitGate extends (ParkingSystem)  
{  
    ExitGate (Display display)  
    { super (display);  
  
    }  
    void notify User ()  
    { display.Show ("Thank you for  
visiting" );  
    }  
}
```

// Usage
ParkingSystem gate = new ExitGate
gate.notifyUser(1, (new LCDisplay()));

◎ Helps with : Separating what
needs to be done from how it's
done.

Interface Display

void show
(String message)

Implements

Class LEP

class LCD

implements

abstract class ParkingSystem

Display display

P.C

abstract void notify
user

Extends

Class Entry

class Exit

P.C

void notifyUser()

{ display.show("T.Y
visibility");

void notifyUser

{ s.v.r("S.P.L. Available");

○ Proxy Pattern :- It controls access to an object. It is a proxy that acts as a placeholder or access controller to another object.

○ When to Use :- To add access control, lazy loading, logging, etc.

eg:- Secure Access to Parking
Lot

```
interface LotAccess
```

```
{  
    void Enter();
```

```
}
```

```
class RealP.L implements LotAccess
```

```
{  
    p.v enter()
```

```
{  
    S.O.P ("Access granted to  
           Parking Lot");
```

```
}
```

```
}
```


class SecurityProxy implements
ListAccess

```
{ RealPL pl = new RealPL();  
  private boolean hasPass;  
  
  SecurityProxy (boolean hasPass)  
  {  
    this.hasPass = hasPass;  
  }  
}
```

P.v enter()

```
{  
    if (hasPass)  
    {  
        pl.enter();  
    }  
    else  
    {  
        S.O.P("No Pass");  
    }  
}
```

// Usage

LotAccess access = new Security
Proxy(true);
access.enter(); // output: Access
granted to Parking lot

© Helps with : Access control and
additional logic before delegating.

Interface Lock

Access

void enter();

Implements

Class Real P.L

enter()

{ S.O.P("Access
Granted"); }

Implements

Class Security Proxy

P.L // object
P.C

P.v enter()
{ if (has Pass)
{ P.L.enter(); }

}
{ S.O.P("No Pass"); }

